Fix without abandoning the per-task loop — pass forward a lightweight manifest, not full regeneration:

python

```
class ProjectManifest(BaseModel):
    files: dict[str, str]   # path -> one-line summary, e.g. "models/user.py: User model (id, email, hashed_password)"
```

Update this after every successful task, and include it in every subsequent `TaskContext` — cheap (a few lines per file, not full content), but gives each task-level call enough shared context to stay consistent without re-reading every file in full.

—----------

Add it as a small field on `CodeAgentResult` (so the same generation call produces its own summary — no extra LLM call needed), then thread it through your orchestration loop as plain in-memory state.

1. Extend your existing schemas — one field each:

```
class CodeAgentResult(BaseModel):
    task_id: int
    success: bool
    files: list[GeneratedFile]
    tests: list[GeneratedFile]
    manifest_entries: dict[str, str]   # {file_path: one-line summary}, model writes this itself
    reflection_notes: str | None = None


class TaskContext(BaseModel):
    task_id: int
```

```
    title: str
    description: str
    document_filters: DocumentFilter
    dependencies: list[int] = []
    manifest: dict[str, str] = {}      # what previous tasks already built, passed in
```

2. Tell the model to produce the summary as part of its normal output — add a line to `prompts.py`'s system prompt: *"For each file you create or modify, include a one-line summary in `manifest_entries` describing its key contents (e.g. class/function names, important fields) — this will be shown to future tasks so they know what already exists."* Cheap to ask for, since it's already looking at the code it just wrote.

3. Orchestration loop — manifest lives as plain in-memory state, updated after each successful task:

```
def run_pipeline(requirement: str, max_iterations: int = 3):
    manifest: dict[str, str] = {}
    revision_note = None

    for iteration in range(1, max_iterations + 1):
        plan = planner_agent.generate_plan(requirement, revision_note=revision_note)
        try:
            execution_order = resolve_execution_order(plan.tasks)
        except ValueError as e:
            revision_note = f"Previous plan failed: {e}"
            continue

        for task_id in execution_order:
            task = next(t for t in plan.tasks if t.task_id == task_id)
            context = TaskContext(
```

```
            task_id=task.task_id,
            title=task.title,
            description=task.description,
            document_filters=task.document_filters,
            dependencies=task.dependencies,
            manifest=manifest,          # <- current state of the project so far
        )
        result = code_gen_agent.generate(context)
        manifest.update(result.manifest_entries)   # <- grows after every task

    # once all tasks done: single sandbox run (per your earlier decision)
    validation_result = validation_agent.validate_project(manifest_to_filepaths(manifest))
    if validation_result.passed:
        return validation_result
    revision_note = f"Validation failed: {validation_result.feedback}"

return escalate_to_human(requirement, revision_note)
```

Why in-memory and not SQLite: the manifest is scoped to one project-generation run — it's rebuilt from scratch every time `run_pipeline` starts, same lifetime as `execution_order` and `revision_note`. Persisting it to SQLite would only matter if you needed to resume a generation run that crashed mid-way, which isn't a today problem.

One thing to inject into the prompt alongside `task.description`: format `context.manifest` as readable text in `prompts.py` before it hits the model — something like:

```
def format_manifest(manifest: dict[str, str]) -> str:
    if not manifest:
        return "No files created yet."
```

```
    return "\n".join(f"- {path}: {summary}" for path, summary in manifest.items())
```

This is what actually goes into the prompt text — the dict itself is just your Python-side data structure.

Let's make it dead simple: since you're already writing `manifest.json`, keep that approach — don't switch to anything fancier tonight. Here's exactly what to do.

The answer to your actual question: yes, Code Gen Agent updates it, right after each task finishes — not before, not the Planner, not some separate process.

Step 1 — one function that reads the file (or starts empty if it doesn't exist yet):

python

```
import json
import os

MANIFEST_PATH = "manifest.json"

def load_manifest() -> dict:
    if not os.path.exists(MANIFEST_PATH):
        return {}
    with open(MANIFEST_PATH, "r") as f:
        return json.load(f)
```

Step 2 — one function that saves it back:

python

```
def save_manifest(manifest: dict):
    with open(MANIFEST_PATH, "w") as f:
```

```
    json.dump(manifest, f, indent=2)
```

Step 3 — after Code Gen Agent finishes ONE task, call these two functions to update it:

python

```
def update_manifest_after_task(result):  # result = your CodeAgentResult
    manifest = load_manifest()
    for file in result.files:
        manifest[file.path] = f"Part of task {result.task_id}"  # simple, just get it working
    save_manifest(manifest)
```

Step 4 — in your main loop, call this right after `generate()` returns, before moving to the next task:

python

```
for task_id in execution_order:
    task = get_task(task_id, plan)
    result = code_gen_agent.generate(task)
    update_manifest_after_task(result)   # <-- THIS is the update step you were missing
```

Step 5 — before generating the NEXT task, load the manifest and pass it in:

python

```
  manifest = load_manifest()
    task.manifest_context = manifest   # or however you're passing context into the prompt
    result = code_gen_agent.generate(task)
```

That's it. That's the whole loop:

1. Load manifest → 2. give it to Code Gen Agent for this task → 3. agent generates → 4. write what it made back into manifest.json → 5. next task repeats from step 1, now seeing what the previous task added.

For tomorrow, don't worry about making the summaries smart or detailed. Even `"Part of task 3"` as the description is fine for a working demo — the *mechanism* (later tasks can see earlier files exist) is what matters, not the quality of the one-line description. You can make it say something smarter later if you have time.

You don't need this to be perfect tonight — you need it to *run* tomorrow. This version will run.